
Requirements Engineering

Oscar Chaparro

In Sprint 0, you implemented an issue someone else wrote.

What did you have to ask the author?

The agent did what the issue said

ISSUE

“Limit the length of budget category names.”

Open

enhancement

No description provided.

With a human developer, ambiguity produces questions.

What the agent built: input silently stops at 50 characters. No message. Existing longer names are cut on the next edit.

- `<Input value={name} onChange={setName} />`

+ `<Input value={name} onChange={setName} maxLength ={50} />`

What the author meant: a visible counter, a clear message at the limit, and existing names left alone.

With an agent, ambiguity produces code.

Agenda

1 What a requirement is

2 Eliciting requirements

3 Writing good specifications

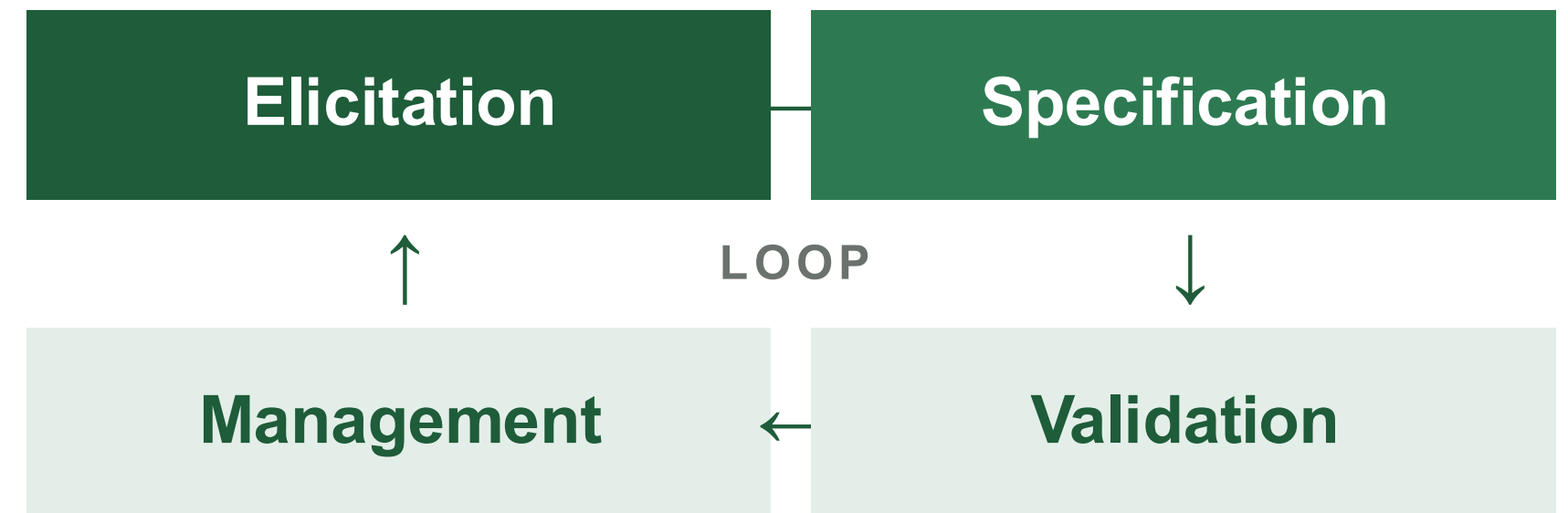
4 Validating requirements

What a requirement is

What vs. how, functional vs. non-functional

Requirement

- A **requirement** states what the system must do, or a quality it must have from a stakeholder's point of view.
- It says **what** and **why**, not **how**.
- **Requirements engineering (RE)** : discovering, writing, checking, and managing requirements.
- Four activities, repeated every sprint: a loop, not a phase.



Requirements engineering in your sprint

Elicitation

feeds D2

Learn current behavior (app, code); read the issue thread and upstream tracker; ask the author, maintainers, or instructor/TA (the customer). Record answers in the issue.

Specification

D2

D2: user story, scenario, acceptance criteria, out of scope, open questions

Management

D1

backlog refinement (D1), decomposing issues, instructor-added priorities

Validation

D2 D4

teammate review of D2; PRs evaluated against the acceptance criteria (D4)

Requirement vs. design/implementation

Requirement (D2)

“Users are told when a category name is too long.”

“Clearing the canvas cannot delete work by accident.”



one
requirement
→ many
possible
designs

Design/implementation decision (D3)

“Add maxLength and a counter to InputCell.”

“Show a confirm dialog using the existing Dialog component.”

Test: a requirement can be met by many different implementations. If the statement already picks one, it's a design/implementation decision.

State the need, not the mechanism

Requirements state conditions stakeholders need. **Design decisions** state how the team meets them.

THE “HOW” AS THE REQUIREMENT

“Add a Redis cache to the search endpoint.”

- Passes as soon as Redis is added
- Search can still be slow — nothing fails
- No alternatives considered (an index?)
- New dependency nobody evaluated

THE NEED AS THE REQUIREMENT

“Search returns results in < 500 ms for 10k documents.”

- Any design must prove the speed
- The team compares options and records its choice in D3

If the “how” is in the requirement, the need is hidden: nobody can check it, and the implementer — human or agent — builds the mechanism, not the outcome.

Need or solution?

A *“Users can find a document by a word in its title.”*

B *“Use a modal to confirm clearing the canvas.”*

C *“Admins can see which users haven’t logged in for 90 days.”*

D *“Store all dates in UTC.”*

Need or solution? For each solution: what is the need behind it?

Need or solution? — answers

A	<i>“Users can find a document by a word in its title.”</i>	Requirement	Functional: says what users can do; how search works is left open.
B	<i>“Use a modal to confirm clearing the canvas.”</i>	Solution	Need: users don’t lose a drawing by accident. Undo might meet it better than a modal.
C	<i>“Admins can see which users haven’t logged in for 90 days.”</i>	Requirement	Functional: who, what, and a clear condition, with no implementation.
D	<i>“Store all dates in UTC.”</i>	Solution	A design/implementation decision (or a project constraint). Need: dates are correct in every time zone.

Functional requirements

Describe **behavior**: inputs, outputs, rules, states, what the user can do.

Gitea

When a repository has no releases, the Releases page shows a message and a link to create one.

Cal.diy

Event type titles longer than the limit are rejected with an error message.

Excalidraw

Clearing the canvas asks for confirmation before deleting elements.

Usually what a user story + acceptance criteria capture.

Business rules

Many requirements are **business rules**: policies of the domain the system must enforce.

Structural

Define domain concepts and the relationships between them.

“An account is checking, savings, or credit.”

“Every account belongs to exactly one user.”

Behavioral

Support business operations: what is allowed, and when.

“If the balance is below the minimum, the user cannot withdraw.”

“Withdrawals over the daily limit are rejected.”

In an existing system: structural rules live in the data schema and models; behavioral rules in validation and code logic.

Non-functional requirements

Quality attributes — **how well** the system does things.

ATTRIBUTE	MEASURABLE EXAMPLE
Performance	“Search returns results in < 500 ms for 10k issues.”
Security	“Only board admins can delete a board; others get 403.”
Usability / accessibility	“Every new control is reachable by keyboard and has an accessible name.”
Reliability	“A failed sync never loses local edits.”
Compatibility	“Budgets from the previous release open without migration errors.”
Maintainability	“New strings go through the i18n function.”

ISO/IEC 25010 quality model

Make it measurable

“The export should be fast.”

“The new page should be accessible.”

“The feature should be secure.”

“Don’t break existing users.”

Rewrite each so someone could check it pass/fail.

Make it measurable — answers

“The export should be fast.”

Performance

Exporting a 1,000-row report finishes in under 3 s on the demo dataset.

“The new page should be accessible.”

Accessibility

All new controls are keyboard-reachable and have accessible names; the *axe* tool reports 0 new violations.

“The feature should be secure.”

Security

Only users with write access see and can call “Delete”; others get no button and a 403.

“Don’t break existing users.”

Compatibility

Data created in the previous release opens without errors; existing tests pass unchanged.

A number or a concrete check makes it testable. Guessed numbers go in Open Questions.

Constraints

Limits on the solution that come from outside the feature.

- Tech stack and frameworks already in the project
- Backward compatibility with existing data and APIs
- Project conventions — CONTRIBUTING.md, linters, i18n, UI component library
- Licenses, dependency policy (“no new deps without discussion”)
- Deadlines — the sprint ends Oct 8

WHERE

CONTRIBUTING.md
your D3 standards doc
AGENTS.md

Constraints limit the design, but stakeholders impose them, so they're requirements.

Why NFRs and constraints get lost

WHY THEY GET LOST

- Nobody states them — “everybody knows.”
- Tests rarely check them.
- Agents optimize for what is stated and tested (L3: intent blindness).

WHERE THEY GO

Project-wide NFRs/constraints

→ CONTRIBUTING.md, AGENTS.md / CLAUDE.md, your standards doc

Issue-specific NFRs

→ an acceptance criterion in the issue

You already know the formats

User story

L4

As a [role], I want [function], so that [value]

3 C's

L4

Card, Conversation, Confirmation

INVEST

L4

split or rewrite stories that fail several letters

Acceptance criteria

L4, glossary

testable pass/fail conditions

Bugs

L4

observed / expected behavior, steps to reproduce

TODAY ADDS

where requirements come from, and what makes them good.

Elicitation

Finding out what is actually needed — in a new system or a system that already exists

Elicitation is discovery

- Needs are **hidden** (tacit), **incomplete**, and **conflicting**.
- Users describe **solutions** and **symptoms** — your job is to find the need.
- Stakeholders assume context you don't have.
- Ambiguity is a normal input, not an error.

what they said

what they need / assume

Stakeholders in open source

End users

use the product

Community

issue threads, discussions, +1s

Self-hosters / operators

run it (Gitea, Discourse, Chatwoot...)

Upstream maintainers

decide what gets merged; set conventions

Your team

builds and maintains your fork

Instructor & TA

the “customer”: may add issues and set priorities

They often disagree: users want features, maintainers want small scope · users want change, self-hosters want stability · the customer’s priorities vs. your plan

Every issue is a change to a system

Example: “Limit the length of budget category names.”

CURRENT BEHAVIOR

Names accept any length.

Found in the app and code — not in the issue.

+

REQUESTED CHANGE

Stop at 50 characters and tell the user.

From the stakeholder, in the issue.

+

MUST NOT CHANGE

Existing longer names still display and can be edited.

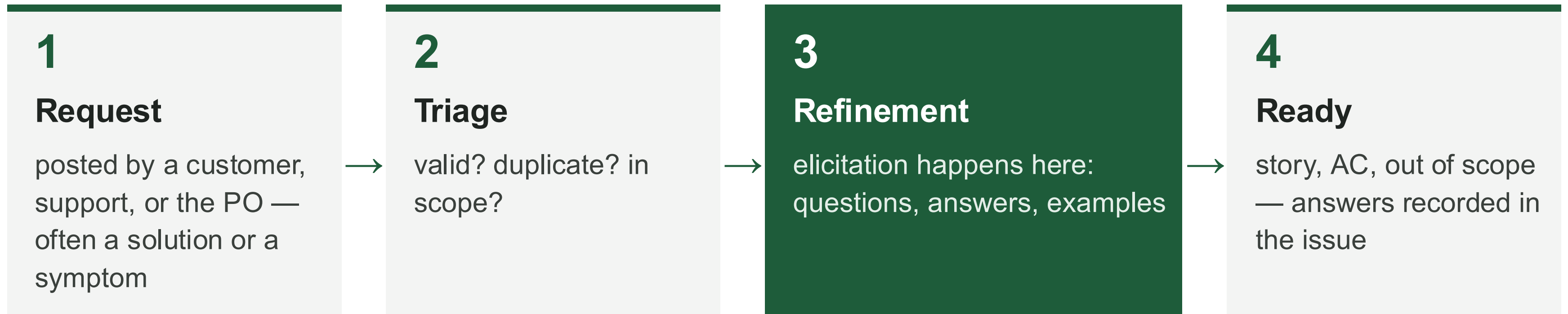
Nobody asks for it; users rely on it.

The spec = acceptance criteria for the change + for what must be preserved.

Where to learn current behavior

SOURCE	WHAT IT TELLS YOU
Running app	current behavior: the de facto spec
Tests	executable requirements: each assertion states expected behavior
Code	validation rules, limits, permissions, edge-case handling
Issue & PR history	what was requested, rejected, and why
Docs, CONTRIBUTING, release notes	intended behavior and rules
Maintainers & discussions	priorities and tacit knowledge

From request to ready issue



OPEN SOURCE

Mostly asynchronous: maintainers ask in the comments, label “needs info”, the requester replies.

COMPANY

Calls with the requester, backlog refinement meetings, quick demos.

The issue records elicitation. It doesn't replace it.

Techniques used in practice

TECHNIQUE

SHARE OF PRACTITIONERS USING IT

Interviews

one-on-one conversation with a stakeholder



Meetings / workshops

facilitated group session, incl. backlog refinement



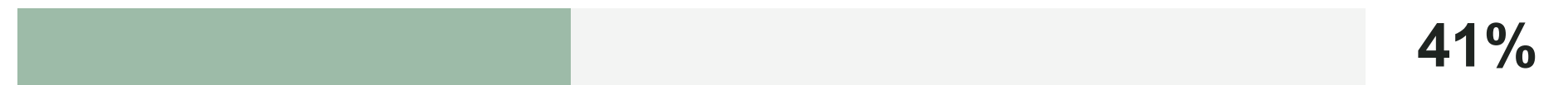
Prototyping

show a sketch, mockup, or demo; collect reactions



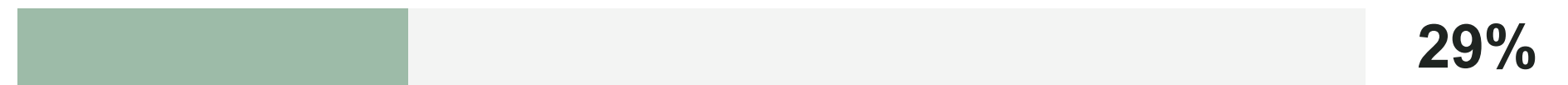
Scenarios

walk through a concrete story of a user doing a task



Observation

watch users do their real work, to see what they don't say



NaPiRE survey, Wagner et al., ACM TOSEM 2019 (N = 228).

Interviews

One-on-one conversation with a stakeholder to learn their needs.

HOW TO RUN IT

- 1 Prepare 5–8 open questions; follow the conversation
- 2 Ask about past behavior: “Tell me about the last time you...”
- 3 Ask “why?” until you reach the need
- 4 Don’t pitch your solution
- 5 Read back what you heard; post it in the issue

EXAMPLE

Issue: “**Add budget alerts.**”

✗ “Would you like budget alerts?”

✓ “**Tell me about the last time you overspent. When did you notice?**”

PITFALL

“Would you...?” questions get polite yeses, not evidence.

Workshops and meetings

A facilitated group session to reach shared understanding and decisions.

HOW TO RUN IT

- 1 One goal, a timebox (~30 mins)
- 2 3–5 people who can answer (product, dev, test)
- 3 Separate facilitator and note-taker
- 4 Use a format, e.g. **Example Mapping**
- 5 End with decisions and open questions recorded

EXAMPLE

Example Mapping — “**Limit category name length**”

RULE	Max 50 characters
EXAMPLE	Paste 60 chars → cut to 50
EXAMPLE	Old 70-char name → still editable
QUESTION	Does the API enforce it too?

PITFALL

Without a facilitator, the loudest voice decides.
Without notes, decisions are lost.

Prototypes

Show a sketch, mockup, or demo and collect reactions.

HOW TO RUN IT

- 1** Pick the question it should answer
- 2** Build the lowest fidelity that answers it: sketch → mockup → branch demo
- 3** Give a task; let the user drive
- 4** Watch where they hesitate; ask why
- 5** Record what you learned as AC; throw it away

EXAMPLE

Two mockups for the category name:

A blocks typing at 50, with a counter **B** allows typing, shows an error on save

The user tries both, picks A, and asks: “What about my old long names?” → AC4

PITFALL

A working demo looks finished: its accidental choices become requirements nobody decided.

Scenarios

Walk through a concrete story of one user doing one task, step by step.

HOW TO RUN IT

- 1 Pick a real user, a goal, and concrete data
- 2 Tell the story one step at a time
- 3 At each step ask “what if...?”: empty, limit, error, other role
- 4 Write the main path and variants as Given / When / Then

EXAMPLE

“Ana renames *Groceries* to a 60-character name and presses Enter.”

What if she pastes it? What if the name already exists? What if she’s offline?

PITFALL

Walking only the happy path.

Observation

Watch users do their real work, in their own setting.

HOW TO RUN IT

- 1** Pick a real task and a real user
- 2** Watch in person or by screen-share; ask them to think aloud
- 3** Note workarounds, hesitations, repeated steps
- 4** Ask about them afterwards, not during
- 5** Record needs, not solutions

EXAMPLE

Watch a user sketch a diagram. Before clicking **Clear**, they take a screenshot “just in case.”

They never mentioned this in the interview; the habit is the evidence.

→ **Need: recover a canvas after clearing it.**

PITFALL

People act differently when watched; one session is one user.

Asking good questions

- Ask about the **need** , not the solution — “What are you trying to do?”
- Ask for **examples and edge cases** — “What should happen if...?”
- Ask what is **out** — “Does this include X?”
- One specific question per comment; answers go **in the issue**

✗ **VAGUE**

“Can you clarify?”

✓ **SPECIFIC**

“When the name is exactly at the limit, should typing be blocked or should we show an error on save?”

What would you ask?

If seated next to a teammate, talk to them. Your issue:

Actual Budget

“Add budget alerts.”

Cal.diy

“Add a waitlist for fully booked time slots.”

Excalidraw

“Let users export only part of the drawing.”

Gitea

“Notify authors when their PR goes stale.”

Medusa

“Support discounts for loyal customers.”

Outline

“Remind owners when documents get outdated.”

Write your three best questions, before any acceptance criteria. Then share one per team.

Agents as elicitation assistants

✓ Good at

- Finding what already exists in code and tests
- Summarizing threads and user feedback
- Drafting questions, edge cases, and Open Questions
- Drafting a story and AC from answers

Keep human

- Asking stakeholders and recording their answers
- Checking every claim and quote
- Resolving conflicts
- Deciding priority and scope

Rule: the agent drafts the questions; stakeholders answer them.

A prompt to prepare questions

Here is issue #123: “Limit the length of budget category names.”

Draft 5 open questions for the issue author about the problem behind it.

- Ask about past situations, not hypotheticals.
- One question per item.

Don’t suggest solutions.

Output: the comment
you post on the issue.

Then edit it: cut what the
code already answers;
add what you know
matters.

Where agents fail at elicitation

AGENTS...

WHAT YOU DO

Retrieve what's written, not what's left unsaid

→ **Ask stakeholders; the agent can't**

Fill gaps silently instead of asking

→ **Ask for questions first, no implementation**

Take a requested solution as the requirement

→ **Ask “what problem does this solve?”**

See only the issue author's view

→ **List who else is affected**

Invent plausible requirements

→ **Check each AC: who asked for it? If no one, ask or cut it.**

Miss tacit knowledge, priorities, politics

→ **Ask the people who know; make trade-offs yourselves**

Lyu et al., “How Do Practitioners Build SE Agents?”, arXiv:2607.10856, 2026.

Specification

Writing requirements someone else can build and test

Qualities of a good requirement

QUALITY	MEANS	X BAD EXAMPLE
Unambiguous	one reading	“show a reasonable limit”
Complete	covers errors and edge cases	no word on existing data
Consistent	no contradictions	AC1 blocks typing, AC3 shows an error on save
Verifiable	pass/fail check exists	“feels responsive”
Feasible	can be built with the constraints	“works offline” in a server-only feature
Necessary & traceable	tied to a need/source	extras nobody asked for

After IEEE 830 / ISO/IEC/IEEE 29148.

Words that signal trouble

fast

easy

intuitive

user-friendly

reasonable

appropriate

flexible

support

handle

manage

improve

and/or

etc.

as needed

**if
possible**

all

never

always

TBD

in an acceptance criterion

Test: would two developers — or two agents — build the same thing?

Find the ambiguities

Limit the length of budget category names

Open

Actual Budget

“When creating or renaming a budget category, the name field accepts any number of characters without restriction. The app should prevent names that exceed a reasonable maximum and inform users when they reach the limit.”

Rewritten

Limit the length of budget category names Open enhancement

STORY

As a **budget user**, I want to know when a category name is too long , so that names display fully in the budget table.

ACCEPTANCE CRITERIA

AC1 Typing stops at 50 characters; pasted text is cut to 50.

AC2 A counter (e.g., “48/50”) appears once 40 characters are typed.

AC3 Renaming follows the same rule as creating.

AC4 Existing names over 50 still display and can be edited down.

OUT OF SCOPE

group names, notes, API-level validation.

OPEN QUESTIONS

Is 50 right? (ask maintainers) Should the API reject longer names?

Scenarios and edge cases

Scenario = one concrete path: **Given / When / Then**

Always ask about:

empty

at the limit

just over

paste vs. type

unicode / emoji

**long translations
(i18n)**

existing data

**other roles and
permissions**

errors and offline

**narrow / mobile
screens**

Use cases

User story

short; starts a conversation

Use case

a structured flow:

- actor
- precondition
- main flow
- alternative flows
- result

Use case: Clear canvas (Excalidraw)

Actor: user **Pre:** canvas has elements

1. User selects “Clear canvas”
2. System asks for confirmation
3. User confirms
4. System removes all elements
5. User can undo

3a. User cancels → nothing changes

Use when a feature has several steps or many alternatives. **Each alternative flow → a test.**

EARS: precise requirements

Ubiquitous always true	The <system> shall <response>. <i>“The category editor shall accept names of up to 50 characters.”</i>
Event triggered	When <trigger>, the <system> shall <response>. <i>“When the name reaches 50 characters, the editor shall show “Maximum 50 characters”.”</i>
State while in a state	While <state>, the <system> shall <response>. <i>“While offline, the app shall save edits locally and sync them when reconnected.”</i>
Unwanted errors, failures	If <condition>, then the <system> shall <response>. <i>“If a sync fails, then the app shall keep local edits and show a Retry option.”</i>
Optional feature present	Where <feature is included>, the <system> shall <response>. <i>“Where email is configured, Gitea shall email the author when their PR goes stale.”</i>

Optional tool: one requirement per sentence, with an explicit trigger and response. Mavin et al., EARS, IEEE RE 2009 (Rolls-Royce).

Validation

Is it the right thing, and is it ready to build?

Validation vs. verification

Validation

Are we building the **right thing**?

Do the spec and the working product match stakeholder needs?

Verification

Are we building the **thing right**?

Does the code meet the spec?



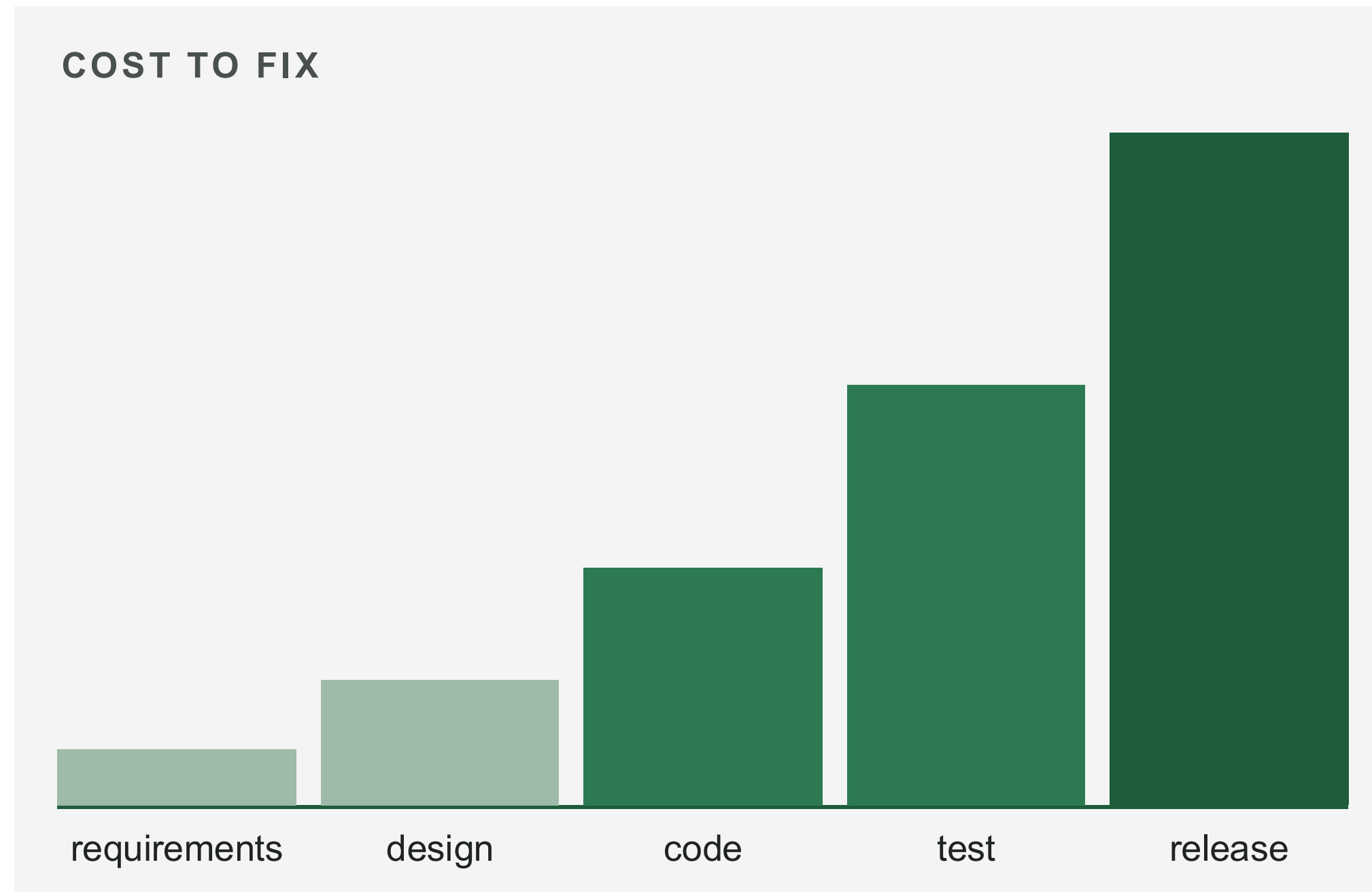
All tests pass = verification.

You can pass every test and still fail validation.

Who decides pass/fail? The spec → verification. The stakeholder → validation.

Validate early

The later a requirements mistake is found, the more it costs to fix.



With agents, spec → code takes minutes.

A wrong spec becomes a lot of wrong code, fast.

Boehm's classic cost-escalation data; exact multipliers are debated, the direction is not.

Validation techniques

Is it right? The stakeholder decides.

TECHNIQUE	HOW	CATCHES
Walkthroughs	step through scenarios with a stakeholder	wrong assumptions
Prototypes / demos	“is this what you meant?”	wrong need

Is it ready to build? A checklist or the current system decides.

TECHNIQUE	HOW	CATCHES
Reviews / inspections	a checklist, a second reader	ambiguity, gaps
Test derivation	write a pass/fail check for each AC	unverifiable AC
Consistency with the current system	does it already exist? conflicts?	invalid issues

Requirements reviews

HAT 1 · IMPLEMENTER

Is it clear and complete?

- current behavior checked (say where)
- edge and error cases
- relevant NFRs and constraints
- out of scope written
- no design decisions in the AC

HAT 2 · TESTER

Is it verifiable?

- each AC is pass/fail and observable
- I can say how I'd check each one

HAT 3 · STAKEHOLDER

Is it the right thing?

- names a real user and a need, not a solution
- open questions asked to someone

Comments are specific questions or rewrites — not “LGTM”.

Author revises; reviewer confirms on the issue.

Review this spec

Gitea — Releases page has no empty state [Open](#)

Story: As a user, I want a message on the Releases page so that it isn't blank.

AC1 The page looks good when there are no releases.

AC2 A helpful message is shown.

AC3 Use the EmptyState template partial.

Write three review comments.

Useful review comments

NOT USEFUL

X “LGTM”

X “Make the AC clearer.”

USEFUL

✓ “AC2: which text? Proposal: ‘No releases yet’ + a ‘New release’ link.”

✓ “What does a user without write access see? Add an AC.”

✓ “AC3 is a design/implementation decision — move it to D3.”

✓ “Tags but no releases: empty state or not?”

From acceptance criteria to checks

For each AC, write one pass/fail check.

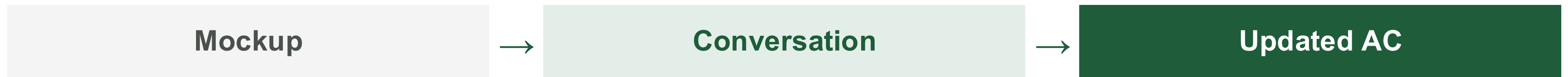
AC	BEHAVIOR	HOW YOU'D CHECK IT
AC1	typing stops at 50; paste cut to 50	type 51 characters, paste 60 → field holds 50
AC2	counter appears at 40	type 39 → no counter; type 40 → counter
AC3	rename follows the same rule	rename to 51 characters → blocked at 50
AC4	existing long names still editable	seed a 70-character name, edit it → saves
AC5	the counter is unobtrusive	✗ no pass/fail check → rewrite

An AC you can't check is an AC you should rewrite.



Prototypes as validation

- Show before you build: a mockup, a screenshot, a quick branch demo.
- Ask: **“Is this what you meant?”**
- With agents, prototypes are almost free → validate more often.
- A prototype is not the spec: record what you learned in the AC, then throw it away.



AI-assisted requirements work

What to delegate, what to own

The bottleneck moves

“Writing code isn’t the slow bit, it never was. It’s gathering requirements, interfacing with the other team, deploying it... the writing of the code is the easy bit.”

— Practitioner P10, Lyu et al. (2026)

“The hardest single part of building a software system is deciding precisely what to build.”

— Brooks, No Silver Bullet (1987)

Where agents help in RE

ACTIVITY	AGENT CAN	HUMAN MUST
Elicitation	read code and tracker, draft questions	ask stakeholders, get answers
Specification	draft story and AC, list edge cases	choose scope, decide the numbers, explain every AC
Validation	check against the checklist	judge if it's the right thing
Management	find duplicates, flag spec–PR drift	decide changes

The agent works from what's written down. Needs, trade-offs and numbers live in stakeholders' heads, and someone has to own them.

When agents write requirements

- ! **Invented requirements** — numbers and behaviors nobody asked for
- ! **Silent gap-filling** — one interpretation becomes code before intent is settled
- ! **Spec inflation** — long, generic AC that look complete
- ! **Same blind spots** — an agent reviewing an agent-written spec tends to agree with itself, so an agent review is never the last review
- ! **Ownership** — you must be able to explain every AC, like every line of code

L3 CALLBACK

Intent debt

goals, constraints and specs that were never captured



Spot the invented requirement

Request: “Show a character counter on the Gitea issue title field.”

AGENT-DRAFTED AC	
1	A counter “n/255” appears under the title field.
2	The counter turns red above 90%.
3	The limit is raised to 500 characters.
4	The same counter is added to PR titles and milestone names.
5	Screen readers announce the count on every keystroke.

Which ones did anyone actually ask for?

A human-agent RE workflow

	ELICIT		SPECIFY	VALIDATE		BUILD	CHECK
Human	1 state intent and context (issue, source)		3 get answers from stakeholders		6 + teammate — judge what the agent can't: right need? right numbers?		8 check against the AC; demo to the stakeholder
Agent		2 READ-ONLY report current behavior, list open questions		4 draft story, AC, out of scope	5 review against the checklist	7 derive checks from AC, then Explore → Plan → Implement (L3)	



Prompt: agent as spec reviewer

You are reviewing a requirements spec, not code. Do not edit files.

Using this checklist: [paste the review checklist]

List: ambiguous words, missing edge or error cases, AC that are not pass/fail, design/implementation decisions inside AC, decisions inside AC, missing NFRs or constraints.

Quote the exact text and propose a rewrite for each.

Do not invent product decisions — list them as open questions instead.

Prompt: acceptance criteria → checks

For each acceptance criterion in issue #<N>:
- write one pass/fail check: the action or input,
and the observable result.
Mark any AC you cannot check this way and explain why.
Do not write tests or code yet.

**AC you can't
can't check
→ rewrite the
AC**

Put specs where the agent reads them

PROJECT LEVEL

AGENTS.md / CLAUDE.md, CONTRIBUTING.md

standing NFRs and constraints

TASK LEVEL

the issue

story, AC, out of scope, open questions

- The agent builds from the spec: a wrong spec gives you wrong code (L3).
- Specs are becoming versioned, tested artifacts, like code (Lyu et al., 2026).

A photograph of a hospital room. A patient is lying in a bed, covered with a white sheet. To the left of the bed is a large, white medical device, a Varian linear accelerator, with the brand name visible. To the right of the bed is a floor lamp with a dark base and a gold-colored shade. The room has a light-colored wall and a dark floor. The image is dimly lit, with the lamp providing the main source of light.

When requirements fail

Therac-25, seen through requirements

Therac-25 (1985–1987)

- Radiation therapy machine built by AECL.
- Six known accidents with massive overdoses; several patients died.
- Lecture 1: the immediate cause was a race condition in the software.
- **Here: the requirements failures that let it reach patients.**

Leveson & Turner, "An Investigation of the Therac-25 Accidents," IEEE Computer, 1993.



Therac-25 through an RE lens

RE FAILURE	WHAT HAPPENED	IN THIS LECTURE
Missing safety requirement	Hardware interlocks were dropped; no requirement said the software alone must prevent an overdose.	NFRs and constraints get lost
Missing error-handling requirements	“MALFUNCTION 54” was undocumented; one key resumed treatment.	missing error cases
Unvalidated assumptions	Reused code relied on hardware that had masked its bugs.	consistency with the current system
Ignored feedback	Hospital reports were met with “an overdose is impossible.”	validating the running code

Lessons for your project

- 1 Safety, security, and data loss are requirements — **write them down.**
- 2 Error messages are requirements: “If X, then the system shall tell the user Y.”
- 3 Reused code carries hidden assumptions — re-check them in the new context.
- 4 User reports are validation data — triage them, don’t dismiss them (L4).
- 5 Agents reuse code and patterns eagerly — ask what the reused code assumes.

Traceability and change

Traceability



HELPS YOU

UNDERSTAND

- Why is this code here?
- Who asked for it?

CHANGE

- Where is feature X implemented? (*concept location*)
- What breaks if it changes? (*impact analysis*)
- Which tests to rerun?

CHECK

- Is every requirement built and tested?
- Is there code nobody asked for?
- What shipped in this release?

You already do most of it: **Closes #N**, AI log links on issues, release descriptions.

Make requirement changes visible

- 30%+ of requirements change during development (L2).
- Edit the issue and add a comment: what changed and why.
- A change needs the stakeholder's OK: validate it again.
- Follow the trace links: update the tests and linked PRs.
- Scope grew? Re-estimate or split — don't stretch.
- Agent extras ("while I was here I also...") are unreviewed requirements → new issue.

ISSUE TIMELINE

- Issue opened: AC2 "counter appears at 40"
- **edited** AC2: 40 → 45
- **comment:** maintainer asked for 45; 40 felt too early

Takeaways

- 1 State the need, not the mechanism.
- 2 Every issue is a change: check current behavior first.
- 3 Elicit with concrete questions, and record the answers in the issue.
- 4 Validate early: the stakeholder decides if it's right; the spec decides if the code is.
- 5 Agents do the legwork; **humans answer the questions and make the decisions** (scope, numbers).
- 6 Write down safety, error and quality requirements, trace them, and keep every change visible.

